functionality or algorithms.

Because Java's `keytool` command doesn't support generating keys for the encryption algorithm that `SecretBox` uses, you can instead generate a standard AES key and then convert it as the two key formats are identical. `SecretBox` only supports 256-bit keys, so run the following command in the root folder of the Natter API project to add a new AES key to the existing keystore:

```
keytool -genseckey -keyalg AES -keysize 256 \
    -alias aes-key -keystore keystore.p12 -storepass changeit
```

You can then load the new key in the `Main` class just as you did for the HMAC key in chapter 5. Open Main.java in your editor and locate the lines that load the HMAC key from the keystore and add a new line to load the AES key:

```
var macKey = keyStore.getKey("hmac-key", keyPassword);    #A
var encKey = keyStore.getKey("aes-key", keyPassword);    #B
```

#A The existing HMAC key
#B The new AES key

You can convert the key into the correct format with the `SecretBox.key()` method, passing in the raw key bytes, which you can get by calling `encKey.getEncoded()`. Open the Main.java file again and update the code that constructs the `TokenController` to convert the key and use it to create an